

INTERNSHIP REPORT

Organization: semanticClimate

Intern: Ali Mehdi Naqvi

Timeline: 23 April 2026 - 30 June 2026

Project Report 1: CABot — Production Async Local RAG Server

Organization: semanticClimate (under the CODATA OT-ViRaRe initiative)

Project Track: Backend Infrastructure Engineering & Production Deployment

Intern: Ali Mehdi Naqvi

Timeline: 4 May 2026 - 1 June 2026

1. Project Overview & Objective

This project objective was to design and deploy **CABot**, a production-grade, highly scalable local RAG chatbot engineered to serve the Climate Academy textbook. The system needed to support up to 100 concurrent student users, perform zero-latency responses, run entirely on localized institutional infrastructure to maintain data sovereignty, and successfully launch to the public.

2. Technical Stack

- **Core Languages & Web Framework:** Python, Flask REST API.
- **Task Management & Queue Broker:** Celery, Redis.
- **Vector Store & Embeddings:** ChromaDB, Sentence Transformers (all-MiniLM-L6-v2).
- **Local Inference Stack:** Ollama running llama3.1:8b.
- **Production Gateway & Web Services:** Gunicorn, Nginx.

3. Architectural Blueprint & System Design

To handle heavy multi-user concurrent traffic without blocking network requests, I decoupled the API execution layer from the LLM inference thread using an asynchronous worker pattern:

- **Non-Blocking REST Endpoints:** Developed a robust routing architecture featuring /health, /session, /chat, /result/<task_id>, and DELETE /session/<id>.
- **Asynchronous Queue Brokerage:** When a student posts a question to the /chat route, the Flask server logs the payload, generates a unique job tracking ID, pushes the job to a Redis broker queue, and responds to the client in under 50ms.
- **Worker Execution Matrix:** Dedicated background Celery workers pull jobs from Redis, run the vector search, issue local inference requests to Ollama, and save the final text generation back to the Redis session store. The frontend client smoothly polls the /result

endpoint to stream the answer.

4. Critical Engineering Challenges & Solutions

A. Flat HTML Document Ingestion Failure

The initial document indexing engine (`html_sectioning.py`) was built assuming standard nested `<section>` tags. The target text (`climate_academy.html`) instead used a completely flat DOM structure delimited strictly by `<h1>` through `<h6>` header markers. This mismatch caused the original script to fall back and group all 129,661 words into a single, massive synthetic chunk, stripping out all search capabilities and breaking document citations.

Solution: I completely rewrote the ingestion script into a stateful heading-boundary parser. This new parser sequentially scans the DOM layout, splitting text on every heading tag to generate 1,108 isolated, context-rich chunks complete with proper decimal citation headers (e.g., §12 — Tipping Points - Social).

B. Network Isolation & Dependency Clashes on HPC Architecture

Deploying to the NIPGR HPC environment presented major constraints due to shared, non-sudo configurations, closed network firewalls, and pre-existing library errors (such as `onnxruntime` failing to build on specific server compute chips).

Solution: I resolved port access limits by establishing persistent, multi-node SSH tunnels to bridge local ports onto dedicated remote A100 GPU compute instances. Platform dependency issues were fixed by adding cross-platform target declarations and strict wheel package overrides directly inside the `pyproject.toml` file via the `uv` toolchain.

5. Deliverables & Technical Milestones

- **Core Application:** Fully functioning async REST backend integrated with a custom HTML/JS interface.
- **Infrastructure Configuration:** Production-ready Gunicorn configs, Nginx reverse proxy routing, and automated `systemd` management service files.
- **Operational Documentation:** Detailed setup and operations documentation, including `LAB_DEPLOY.md`, `DEPLOY.md`, and an automated cluster orchestration script (`start_all.sh`).

Project Report 2: Educational RAG Architecture & Baseline Notebook

Project Track: Curriculum Development & Baseline Prototyping

Intern: Ali Mehdi Naqvi

Timeline: 2 June 2026 - 5 June 2026

1. Project Overview & Objective

The goal of this project was to design a clean, scratch-implemented Retrieval-Augmented Generation (RAG) notebook inside Google Colab. This artifact serves as a step-by-step teaching guide for students and researchers to learn the mechanics of document ingestion, parsing, embedding generation, vector databases, and conversational synthesis.

2. Technical Stack

- **Runtime Environment:** Google Colab
- **Language:** Python
- **Document Parsing:** pdfplumber
- **Vector Storage:** ChromaDB
- **Embedding Model:** Sentence Transformers (all-MiniLM-L6-v2)
- **Inference Pipeline:** Groq API
- **Orchestration:** LangChain (RecursiveCharacterTextSplitter framework)

3. Implementation Details & Architecture

The notebook was constructed sequentially across 8 modular phases to maximize transparency and pedagogical clarity:

1. **Environment Setup:** Installing isolated package groups and handling pre-existing dependency warnings safely.
2. **Secure Gateway Hooks:** Incorporating getpass mechanics to handle active API keys securely without hardcoding credentials into public code cells.
3. **Layout-Aware Document Ingestion:** Parsing arbitrary input formats (PDF, DOCX, TXT) into raw text string blocks.
4. **Pedagogical Text Chunking:** Utilizing recursive text splitting parameters, teaching users the explicit structural tradeoffs between chunk size and sliding token overlaps.
5. **Vector Database Loading:** Serializing text chunks into dense vector representations and mounting them inside a local ChromaDB instance.
6. **Similarity Retrieval:** Running cosine distance queries to pull top candidate context segments matching a user's prompt.
7. **Context-Grounded Generation:** Compiling the retrieved context with the prompt and routing it to the LLM.
8. **Conversational QA Memory Loop:** Managing multi-turn conversation states to maintain thread relevance across consecutive queries.

4. Verification & Testing

The notebook was systematically verified against two major test datasets:

- **Curriculum Vitae Target:** Evaluated with johndoe_Resume.docx, yielding 10 clean text chunks and achieving a 100% target accuracy rate across all test QA checks.
- **Climate Science Corpus:** Tested with a massive text file (climate_academy.txt), parsing out exactly 2,568 chunks. The architecture successfully maintained a 5-turn deep conversation loop and demonstrated strict boundary enforcement. For example, when prompted with an out-of-scope question ("What is the latest iPhone?"), the system correctly refused to answer, validating the grounding constraints.

5. Limitations & Engineering Key Takeaways

While highly successful as an educational baseline, this development cycle highlighted severe architectural limitations:

- **Data Sovereignty Violations:** Relied entirely on external cloud APIs, making it unusable for corporate or institutional environments handling sensitive data.
- **Monolithic Weakness:** The linear notebook structure lacked asynchronous processing or task management, meaning concurrent user loads would completely stall execution.
- **Rate Dependencies:** The runtime remained bound to free-tier cloud constraints, pointing to the critical need for a localized, production-hardened backend server.

Project Report 3: semantic_RARe — Multi-Doc GraphRAG CLI Tool

(under the CODATA OT-ViRaRe initiative)

Project Track: Advanced Knowledge Graph Engineering & Research Assessment

Intern: Ali Mehdi Naqvi

Timeline: 5 June 2026 - 30 June 2026

1. Project Overview & Objective

Current academic and institutional evaluation frameworks remain heavily reliant on simple, quantitative metrics, primarily ranking organizations and scholars based on their raw volume of publications. This methodology introduces significant bias by failing to account for the actual real-world impact, systemic value, and semantic magnitude of the research produced.

Driven by the need to shift toward qualitative, substance-based evaluation metrics, the motivation for **semantic_RARe** is to engineer a localized GraphRAG pipeline capable of deeply auditing and extracting semantic magnitude data directly from dense Institutional Annual Reports. By shifting the paradigm from static publication counting to mapping relational impact networks, this tool represents a foundational step toward modernized, open-science research assessment frameworks under CODATA standards.

2. Technical Stack

- **Interface Architecture:** Typer, Rich (providing interactive terminal UIs).
- **Document Parsing Engine:** pdfplumber (configured for layout-aware table extraction).
- **Knowledge Store Matrix:** NetworkX (Directed Knowledge Graphs) paired with ChromaDB (Vector Search Space).
- **Extraction & Logic Layer:** Groq API (Llama-3.1-8b-instant).
- **Graph Visualization Handshake:** GraphML exports designed for Cytoscape.

3. Key Architecture & Engineering Innovations

- **Asynchronous Ingestion Flow:** Engineered a non-blocking UI framework that processes heavy document extractions in the background, keeping the terminal responsive.
- **Dynamic Scope Purity Filtering:** Built an isolated runtime filtering mechanism controlled by CLI arguments (e.g., `--name nipgr`). This feature instantly blocks structural data contamination by ensuring the parsing engine only records relationships matching the targeted facility.
- **Semantic Entity Deduplication:** Created a consolidation utility that automatically merges repeating references to identical entities while filtering administrative noise—such as stripping internal rank levels while retaining highly valued external fellowship milestones.
- **Two-Pass Grounding Verification:** Designed a strict validation loop to combat hallucination risks. The engine routes the generated response and cross-references it against deterministic relational paths within the local knowledge graph, triggering a hard refusal block if any unverified claim is caught.

4. Operational Validation: NIPGR Case Study

The system was evaluated using a multi-institutional umbrella report to assess performance:

- **Ingestion Metrics:** Successfully processed the target text, extracting 1,647 unique relational graph edges mapped cleanly into a local data file (nipgr_graph.json).
- **Target Data Recovery:** Extracted dense relationship webs outlining the BioNEST incubator program, cross-facility grants (DBT, DST, ICAR, Wellcome Trust), proprietary crop intellectual property (SAATVIK chickpea), and prestigious research fellowship appointments.
- **Temporal Tracking Capabilities:** Enabled cross-year analytical mapping using a custom `--year`` metadata tag, allowing researchers to evaluate institutional growth trends across a 10-year tracking period.

5. Project Deliverables & Artifacts

- **Core Application:** Production-ready Python CLI utility engineered with structured commands (ingest, query, temporal-report).
- **Data Artifacts:** Comprehensive data packages containing structural graphs (nipgr_graph.json), chronological transaction history logs (chat_log.jsonl), and standard markdown summaries (assessment_report.md).
- **Visualization Assets:** Stylized .graphml files ready for direct import into Cytoscape, pre-configured with node sizing based on degree centrality and color-coded entity groups.